SNOWFLAKE MIGRATION PROJECTS

Top 10 Interview Cross-Questions with Detailed, Interview-Ready Answers

Projects covered

- SQL Server to Snowflake migration using Matillion
- S3 / ADLS ingestion and Snowflake Medallion architecture (Bronze, Silver, Gold)

How to use this guide

Each answer is written in first person so you can practice speaking it in an interview. Adapt the details to match what you actually built. Where the guide mentions alternatives, explain the option your project used and why.

Interview principle: explain the business requirement, the design choice, the implementation, validation, failure handling, and trade-offs. Avoid claiming metrics or tools you cannot substantiate.

Contents

- 1. Large historical backfill alongside incremental loads
- 2. Migrating T-SQL logic and stored procedures
- 3. Source-to-target data reconciliation
- 4. Incremental loading without a reliable timestamp
- 5. Restarting failed Matillion jobs safely
- 6. Choosing COPY INTO, Snowpipe, and Matillion
- 7. Incremental processing across Medallion layers
- 8. SCD Type 2 and multiple changes in a batch
- 9. Troubleshooting a slow Snowflake transformation
- 10. Production cutover and rollback

1. How would you migrate 2–3 TB of historical data without impacting ongoing incremental loads?

What the interviewer is testing: Migration planning, parallel extraction, bulk loading, orchestration, consistency controls.

Interview-ready answer

I would separate the migration into a controlled historical backfill and a continuously captured delta stream, rather than trying to move everything in one long-running job.

First, profile and partition the source. I would identify the largest tables, their primary keys, date ranges, indexes, change patterns, and dependencies. For large tables, I would divide extraction into non-overlapping chunks—for example, by a stable key range or date range—so multiple Matillion jobs can run in parallel without reading the same records.

Second, load in bulk. I would extract chunks to staged files, typically in cloud object storage where appropriate, and load them into Snowflake using bulk-loading patterns such as COPY INTO. I would tune parallelism based on source SQL Server capacity, network throughput, file sizes, and Snowflake warehouse capacity, not simply maximize the number of threads.

Third, capture changes during the backfill. If the source supports SQL Server CDC or Change Tracking, I would use it to capture inserts, updates, and deletes while the historical copy runs. Otherwise, I would use a reliable watermark strategy or a controlled re-read of overlapping ranges, depending on source capabilities. The exact strategy must account for updates to old records and deletes, not just newly inserted rows.

Fourth, reconcile and apply deltas. I would load historical data into a controlled landing area, validate each chunk, then apply captured changes in sequence using deterministic merge logic. Before cutover, I would run a final delta catch-up and validate key counts and business aggregates.

Concrete example to use if asked: I would maintain a migration control table with table name, chunk/range, extraction start and end, row count, load status, source watermark, target load timestamp, and error details. A chunk is marked complete only after loading and validation succeed.

Likely follow-up probes

- How do you avoid a gap between the historical snapshot and CDC capture?
- What if a row is updated while its historical chunk is being extracted?
- How do you handle deletes?
- What do you do if one chunk fails while other chunks have succeeded?

Common weak answer to avoid: Do not say only “I used parallelism.” Explain chunk boundaries, consistency, change capture, restartability, and how you know every source change is represented.

2. How did you migrate SQL Server stored procedures, functions, and complex T-SQL logic to Snowflake?

What the interviewer is testing: SQL dialect knowledge, modernization judgment, testing discipline.

Interview-ready answer

I would not treat the migration as a direct copy-and-paste exercise. I would inventory stored procedures, functions, views, SQL Agent jobs, temporary tables, dynamic SQL, and dependencies, then classify each object by complexity and business criticality.

Next, I would map incompatible constructs. T-SQL-specific syntax, data types, date functions, identity generation, error handling, procedural loops, temporary-table behavior, and transaction assumptions need individual review. Simple

transformations can often become Snowflake SQL views or DML; more procedural workflows may need Snowflake Scripting or another appropriate implementation.

Then I would separate orchestration from transformation. Matillion orchestration jobs can manage sequencing, dependencies, parameters, and failure paths. Matillion transformation jobs or Snowflake SQL can perform set-based transformations close to the data. I would avoid moving large data sets out of Snowflake just to process them externally.

Finally, I would validate behavior, not just syntax. I would test representative edge cases—NULL handling, time zones, decimal precision, duplicate keys, date boundaries, and error paths—and compare outputs against the SQL Server implementation.

Concrete example to use if asked: For a legacy stored procedure that loops over rows, I would first determine whether its logic can be rewritten as set-based SQL. If procedural control is genuinely required, I would use an appropriate Snowflake scripting pattern and keep orchestration responsibilities clear.

Likely follow-up probes

- How would you replace SQL Server IDENTITY behavior?
- What are the risks when converting DATETIME and DECIMAL columns?
- Would you move every stored procedure into Matillion?
- How would you prove that a rewritten procedure produces equivalent results?

Common weak answer to avoid: Avoid claiming all T-SQL syntax is supported unchanged. The best answer describes an inventory, compatibility mapping, redesign where needed, and regression testing.

3. How did you validate that migrated data was correct—not just that row counts matched?

What the interviewer is testing: Reconciliation strategy, data quality, root-cause analysis.

Interview-ready answer

I would use multiple levels of reconciliation because a matching row count alone does not prove correctness.

Level 1: structural checks. Compare table and column coverage, data types, nullability assumptions, and key constraints or their Snowflake equivalents.

Level 2: volume and distribution checks. Compare row counts by table and meaningful partitions, such as business date, region, or source system. This helps localize a mismatch instead of comparing only the full table.

Level 3: business aggregates. Compare sums, minimum and maximum dates, distinct business keys, null counts, and key financial or operational metrics. Use consistent filters, decimal precision, and time-zone assumptions.

Level 4: record-level checks. For critical tables, compare primary/business keys and normalized column values, or use hashes over consistently normalized fields. Hashes are a diagnostic aid, not a substitute for defining NULL, formatting, and precision rules correctly.

Level 5: business validation. Ask data owners to validate key reports and business scenarios before sign-off. Record exceptions and their resolution in a reconciliation log.

Concrete example to use if asked: If total row counts match but revenue differs, I would compare revenue by date and business unit, identify the first mismatching partition, compare keys within it, and inspect source/target values for rounding, duplicate joins, filtering, or late-arriving updates.

Likely follow-up probes

- How do you compare billions of rows efficiently?
- What if source and target use different decimal precision?
- How do you handle records changing during reconciliation?

- What evidence is required for business sign-off?

Common weak answer to avoid: A credible answer distinguishes technical reconciliation from business acceptance and explains how mismatches are isolated.

4. How would you implement incremental loading if the source has no reliable LAST_UPDATED_DATE column?

What the interviewer is testing: CDC options, watermark limitations, correctness under updates/deletes.

Interview-ready answer

I would first confirm whether the source exposes another reliable change mechanism. I would not assume that a primary key or creation timestamp can detect updates and deletes.

Preferred option: If enabled and available, use SQL Server CDC or Change Tracking according to the source's requirements and retention settings. CDC can provide change records; Change Tracking identifies changed keys and requires appropriate source reads to retrieve current values.

Alternative options: Use a source audit/change table, transaction log-based capture through an approved tool, or a periodic comparison of source and target keys and attributes. For smaller tables, a full reload may be safer and simpler. A hash comparison can help detect changed records, but the source still needs to be scanned or otherwise enumerated.

If a timestamp exists but is imperfect: Use an overlap window and deduplicate deterministically, then reconcile late updates. This reduces risk but does not guarantee correctness if old timestamps can change arbitrarily.

For deletes: Capture explicit delete events or compare key sets. A timestamp-based extract alone normally cannot detect hard deletes.

Concrete example to use if asked: I would store the last successfully committed source position or watermark only after the batch is loaded, merged, and validated. The process should be idempotent so replaying the same change range does not create duplicate target rows.

Likely follow-up probes

- What is the difference between SQL Server CDC and Change Tracking?
- What if the watermark is advanced before the target merge succeeds?
- How would you detect hard deletes?
- What if an old record changes but its timestamp is not updated?

Common weak answer to avoid: Do not promise a perfect incremental process without a trustworthy change signal. Be explicit about the limitations and the reconciliation mechanism.

5. A Matillion job fails after loading half the data. How do you restart it without duplicates or corruption?

What the interviewer is testing: Idempotency, transaction boundaries, audit design, recovery.

Interview-ready answer

I would first identify which steps completed and which data was committed. I would not blindly rerun the whole workflow until I understand whether the destination contains a partial load.

Use batch-level control. Assign a batch ID or load ID and record source range/file, start time, status, row counts, and error details in an audit table. Use statuses such as STARTED, LOADED, VALIDATED, and COMPLETED.

Land first, then publish. Where practical, load into a staging or landing table, validate the batch, deduplicate it, and then apply it to the target using deterministic merge logic. Keep the target update separate from the ingestion bookkeeping so the recovery process can detect a load that succeeded but whose audit update failed.

Make retries idempotent. Reprocessing the same file or change range should produce the same target state. Use stable business keys, source version/change ordering, file-level tracking, and deduplication. Do not rely solely on a job status flag to prevent duplicates.

Design recovery by step. If a file was loaded but the audit update failed, check the staged/load history and target contents before deciding whether to reload. If a transformation failed, rerun from the validated landing stage when possible. Snowflake transactions can help group supported DML operations, but they do not make an entire multi-system Matillion workflow atomic.

Concrete example to use if asked: For a file-driven load, I would track file name plus an appropriate file identity, load it to landing, validate it, and merge by business key. The audit record is updated to complete only after validation and target application succeed.

Likely follow-up probes

- What if COPY INTO succeeds but the next Matillion component fails?
- How do you distinguish a retry from a new source delivery with the same filename?
- What if a batch contains duplicate business keys?
- Which operations can be wrapped in a Snowflake transaction?

Common weak answer to avoid: Avoid saying “Matillion automatically handles everything.” Explain what state is persisted, what is safe to retry, and how partial success is detected.

6. Why choose COPY INTO, Snowpipe, or Matillion loading components? Why not use only one?

What the interviewer is testing: Ingestion design, latency/cost trade-offs, understanding component responsibilities.

Interview-ready answer

I would choose based on data volume, arrival pattern, freshness SLA, file characteristics, and operational complexity. These options can coexist because they solve related but different parts of the ingestion problem.

COPY INTO: Best suited to controlled bulk loads from staged files, such as large historical backfills or scheduled batches. It provides explicit control over when a load runs and is a natural fit for backfill jobs.

Snowpipe: Useful when new files should be ingested automatically with low latency after arrival. It reduces the need to schedule frequent polling jobs, but it has its own ingestion costs and operational considerations. It is not the default choice for every large backfill.

Matillion: Provides orchestration, source connectivity, transformation components, dependencies, parameters, and error handling. It can coordinate loading and transformations; it does not replace the need to choose the right Snowflake loading mechanism for each workload.

Small-file scenario: If thousands of tiny files arrive frequently, I would examine file sizes and arrival patterns, consolidate files upstream when feasible, and avoid excessive per-file overhead. I would measure end-to-end latency and cost before selecting an approach.

Concrete example to use if asked: For a multi-terabyte initial load, I would normally favor a bulk-loading pattern. For frequent incremental file arrivals where freshness matters, I would evaluate Snowpipe. Matillion can orchestrate either approach and the downstream transformations.

Likely follow-up probes

- What file sizes and arrival patterns would influence your decision?
- Does Snowpipe guarantee that every file is processed instantly?
- How do you handle malformed files?
- How do you prevent a file from being loaded twice?

Common weak answer to avoid: Do not describe Snowpipe as a transformation engine or claim it is always faster or cheaper. Explain the workload-specific trade-off.

7. How do you process incremental changes across Bronze, Silver, and Gold? What if an old record changes?

What the interviewer is testing: Medallion architecture, change propagation, freshness and replay design.

Interview-ready answer

I define a clear responsibility for each layer and make the movement of changes between layers explicit.

Bronze: Preserve source-aligned data and ingestion metadata, such as source file, load timestamp, batch ID, and source version where available. The layer should support traceability and controlled replay.

Silver: Apply schema normalization, type casting, data quality rules, deduplication, and business-key standardization. For mutable entities, use deterministic incremental merge logic. For late updates, retain enough source change information to identify and apply the correction.

Gold: Build facts, dimensions, aggregates, and business marts. Propagate relevant changes from Silver into Gold while respecting fact/dimension dependencies and business definitions. For historical dimension changes, use the appropriate SCD strategy.

Choose the processing mechanism deliberately. Streams and Tasks can support explicit change-driven processing and scheduling. Dynamic Tables can express transformations declaratively with target-lag/freshness requirements and dependency management. They have different operational semantics, so I would select based on transformation support, latency, control requirements, and cost.

Late-arriving or corrected data: Reprocess the affected keys or partitions, and ensure downstream models can be corrected. A pipeline should not assume that all records arrive in chronological order.

Concrete example to use if asked: If a customer address changes after the original order has landed, Bronze retains the source event or snapshot, Silver applies the latest valid customer state, and Gold updates the dimension according to the agreed SCD policy. Facts must use the correct dimension key based on the business's historical reporting requirement.

Likely follow-up probes

- When would you use a Stream and Task rather than a Dynamic Table?
- How do you handle schema drift in incoming JSON or Parquet?
- What does target lag mean, and does it guarantee an exact refresh time?
- How do you replay a failed downstream layer without re-ingesting the source?

Common weak answer to avoid: Avoid presenting all mechanisms as mandatory. Be clear about which layer owns raw retention, quality rules, business logic, and history.

8. How would you implement SCD Type 2 if the same customer arrives multiple times in one batch?

What the interviewer is testing: Dimensional modeling, deduplication, effective dating, deterministic processing.

Interview-ready answer

First, I would clarify the source's change semantics and the business key. SCD Type 2 is intended to preserve history when tracked attributes change, so the design must define what counts as a change and how effective time is assigned.

Deduplicate and order incoming changes. If multiple records for one customer arrive in the same batch, I would use a reliable source sequence, change timestamp, or event/version ID to order them. If the order cannot be determined, I would not invent one; I would quarantine the ambiguous records or agree on a deterministic business rule.

Compare tracked attributes. Compare normalized values or a hash of tracked attributes against the current dimension record. Ignore technical metadata that should not trigger a new version.

Close and insert versions. When a real change is detected, expire the current row by setting its end timestamp and current flag appropriately, then insert the new version with a new surrogate key, effective start, open-ended end, and current flag. The exact timestamp boundary convention should be consistent.

Handle multiple changes in one batch. If the requirement is to preserve every intermediate change, process ordered events sequentially or use a set-based pattern that creates a version for each valid change. If only the final state is required, collapse the batch to the latest record and document that intermediate states are intentionally not preserved.

Protect integrity. Ensure only one current row exists per business key, handle late-arriving changes according to the agreed policy, and make reruns idempotent.

Concrete example to use if asked: For a customer whose city changes from Pune to Delhi and then Bengaluru in the same batch, I would preserve all three states only if the source provides reliable ordering and the business needs event-level history. Otherwise, I would apply the agreed latest-state policy rather than inventing an order.

Likely follow-up probes

- What columns would you include in a Type 2 dimension?
- How would you handle a late-arriving correction whose effective date predates the current version?
- How do you ensure there is only one current row per business key?
- How is SCD Type 1 different?

Common weak answer to avoid: A basic MERGE may not preserve multiple intermediate versions by itself. Explain source deduplication, ordering, and the business decision about historical fidelity.

9. A Snowflake transformation has slowed from 10 minutes to 60 minutes. How do you troubleshoot it?

What the interviewer is testing: Query Profile, pruning, join behavior, spilling, compute versus SQL diagnosis.

Interview-ready answer

I would start with evidence rather than immediately increasing the warehouse size.

1. Confirm what changed. Compare query text and parameters, input row counts, data volume, execution frequency, warehouse size, concurrency, and the time period when the slowdown began. Determine whether the same query is being compared under similar conditions.

2. Inspect Query Profile. Look for expensive table scans, poor micro-partition pruning, large joins, exploding row counts, sorts, aggregations, and local or remote spilling. Check whether time is spent waiting for warehouse resources or executing the query.

3. Validate the logic. Check join keys and cardinality, accidental many-to-many joins, unnecessary columns, repeated scans, non-selective filters, implicit casts, and transformations that prevent effective pruning. Push filters earlier when semantically correct and avoid processing data that is not required.

4. Evaluate compute and concurrency. If the query is compute-bound, test a larger warehouse and compare runtime and credits. If concurrent workloads are competing, isolate workloads or evaluate multi-cluster scaling for concurrency. More compute will not necessarily fix inefficient SQL or poor data access patterns.

5. Re-test and measure. Compare elapsed time, bytes scanned, rows processed, spill, and credits consumed. Use controlled tests and account for cache effects. Consider clustering only when the workload and data justify its maintenance cost.

Concrete example to use if asked: If a join unexpectedly multiplies rows, increasing warehouse size may make it finish sooner but will not fix the incorrect result or the underlying join explosion. I would validate join cardinality and query profile before tuning compute.

Likely follow-up probes

- What is micro-partition pruning?
- When would you consider a clustering key?
- What is the difference between local and remote spilling?
- When would a multi-cluster warehouse help—and when would it not?

Common weak answer to avoid: Avoid jumping straight to clustering or a larger warehouse. Diagnose the bottleneck, validate correctness, then benchmark the least costly effective fix.

10. How would you plan production cutover from SQL Server to Snowflake and define rollback?

What the interviewer is testing: Release planning, source consistency, reconciliation, risk management.

Interview-ready answer

I would treat cutover as a controlled release with explicit entry and exit criteria, rather than simply changing a connection string.

Before cutover: Complete migration and regression tests, validate security and connectivity, confirm job schedules and downstream dependencies, establish performance baselines, and agree on reconciliation tolerances and business sign-off. Run source and target in parallel where practical.

Freeze or capture changes: Define how writes to SQL Server are handled during the final synchronization window. Depending on the system, that may mean a brief write freeze, CDC catch-up, or a controlled dual-run design. Apply all final changes and verify the source position or watermark before switching consumers.

Validate the release: Reconcile critical tables and business metrics, verify row-level access and masking, run critical reports, check job completion, and confirm operational monitoring and support ownership.

Switch consumers gradually where possible: Update connection/configuration or routing, monitor error rates, freshness, query performance, and business metrics, and retain a documented decision point for rollback.

Rollback strategy: Preserve the old SQL Server environment and its operational readiness for an agreed period. Define the trigger for rollback, the owner authorized to decide, how new writes are handled, and how consistency is maintained if any writes occur after cutover. A rollback is not safe unless post-cutover data divergence has been considered.

Concrete example to use if asked: If the final reconciliation finds missing transactions, I would not switch the reporting workload. I would identify the missing source range, apply the delta, rerun validation, and proceed only after the agreed criteria pass.

Likely follow-up probes

- What if SQL Server receives transactions during final reconciliation?
- What if reports are correct but Snowflake performance misses the SLA?
- How long would you retain the old platform?
- Who signs off on business reconciliation and who authorizes rollback?

Common weak answer to avoid: Do not claim rollback is simply “point the application back.” Explain post-cutover writes, data divergence, decision criteria, and ownership.

Rapid Revision: Framework for Any Migration Cross-Question

Use this six-step structure when the interviewer asks a scenario question:

- **Requirement:** What business or technical problem had to be solved?
- **Options:** Which viable approaches did you consider?
- **Decision:** Why did you choose this design given volume, latency, cost, and maintainability?
- **Implementation:** Which Matillion jobs, Snowflake objects, SQL patterns, and controls did you use?
- **Validation and recovery:** How did you prove correctness, detect failure, and retry safely?
- **Outcome:** What improved, and how was that improvement measured?

Snowflake and Matillion terms worth revising

- Matillion orchestration jobs versus transformation jobs; job variables, grid variables, dependencies, and error paths.
- Snowflake stages, storage integrations, file formats, COPY INTO, load history, and Snowpipe.
- Incremental patterns: CDC/Change Tracking where available, watermarks, overlap windows, MERGE, deduplication, and delete handling.
- Medallion design: Bronze traceability, Silver quality/conformance, Gold dimensional models and business metrics.
- Streams and Tasks versus Dynamic Tables; freshness requirements and operational trade-offs.
- SCD Type 1/Type 2, late-arriving data, surrogate keys, effective timestamps, and current-row integrity.
- RBAC, masking and row access policies, warehouse sizing, auto-suspend/resume, Query Profile, and resource monitoring.

Important interview integrity note

Use these answers as a speaking framework, not as a claim that every technique was used in your project. Replace examples with your actual source systems, job names, volume, failure scenarios, validation approach, and measured outcomes. If you did not implement a feature yourself, say how you would approach it rather than presenting it as hands-on experience.